

Сравнение методов поиска решений

Влияние обучения модели на примерах ошибок и их коррекций на эффективность в пространствах с ветвящимися путями

Екатерина Крылова · защита проекта

Задача: арифметические пазлы Countdown-типа

Дан набор чисел $N = \{n_1, \dots, n_m\}$ и целевое значение t . Нужно построить выражение из операций $+$ $-$ $\times$ $\div$, которое равно t и использует **каждое число ровно один раз**.

$$N = \{2, 3, 7, 10\}, \quad t = 42 \rightarrow 10 \times (7 - 3) + 2 = 42 \quad \checkmark$$

Целевая функция: $f(x) = 1 \Leftrightarrow eval(x) = t \wedge uses_all(x, N), \quad f(x) \in \{0, 1\}$

Бинарна и недифференцируема — нет градиента, нет «направления спуска». Решение либо точное, либо никакое.

Пространство поиска — ветвящееся дерево

- Решение строится **пошагово**: на каждом шаге — пара чисел + операция. Наивное дерево растёт экспоненциально: уже для 7 чисел это порядок миллионов ветвей, в зависимости от учёта эквивалентных выражений.
- Большинство ветвей — **тупики**: из текущего состояния значение t уже недостижимо.
- Правильные решения — **редкие листья**, разбросанные без регулярной структуры.

Размер пространства растёт быстрее любого перебора → нужна **эвристика**, которая ведёт поиск к редким правильным листьям.

⇒ Дискретная комбинаторная оптимизация

Класс задач: **SAT · TSP · scheduling**. Применимые методы — те же, что мы сравниваем: жадный спуск, random restarts, поиск с возвратом (backtracking).

Модель — это обученная эвристика поиска

Обученная модель работает как **эвристика выбора следующего шага**: по числам и цели она генерирует одну траекторию поиска. Качество эвристики — доля правильных решений за один проход.

TRAIN-TIME — НЕПРЕРЫВНАЯ ОПТИМИЗАЦИЯ

Обучение = невыпуклая минимизация cross-entropy методом **AdamW** (SGD первого порядка с адаптивным шагом). Проще: модель учится чаще выбирать шаги, которые встречались в успешных траекториях.

INFERENCE-TIME — ДИСКРЕТНАЯ ОПТИМИЗАЦИЯ

Поверх эвристики — поиск в дискретном пространстве: **greedy · majority vote · random restarts · backtracking**.

Вопрос проекта: как качество *train-time* обучения влияет на эффективность *inference-time* поиска?

Учить модель на ошибках и их коррекциях

Коррекционный трейс — запись ветвящегося поиска в линейном виде: модель идёт по ветви, упирается в тупик, ставит токен *<backtrack>* и пробует другую.

Она **видит тупик в контексте** — учится не только правильному пути, но и тому, как выглядит ошибка и как из неё выйти.

Backtracking встроен **внутри одной траектории** — модель ищет с возвратом без N независимых перезапусков.

```
START: 2 3 7 10 → t = 42
10 + 7 = 17      остаются {17, 3, 2}
17 + 3 = 20      {20, 2}    тупик ✗
<backtrack>
17 × 2 = 34      {34, 3}    тупик ✗
<backtrack>
7 - 3 = 4        остаются {4, 10, 2}
10 × 4 = 40      {40, 2}
40 + 2 = 42      ✓
```


Две модели — различие только в данных обучения

МОДЕЛЬ	ДАННЫЕ ОБУЧЕНИЯ	ЧТО УМЕЕТ
exp_a clean	100% правильных решений (~19k трасс)	строит одну траекторию к ответу, без возвратов
exp_f backtracking	70% правильных + 30% коррекционных (~28k)	при тупике генерирует <i><backtrack></i> и пробует другую ветвь

Обе — LoRA-дообучение **Qwen3-8B**. Архитектура, гиперпараметры, бюджет обучения — идентичны. Меняется **только состав данных**.

Подводные камни (результат сам по себе):

- Коррекции строятся из **реальных ошибок самой exp_a**, а не синтетических тупиков.
- Ветви собраны в **trie**; точка *<backtrack>* ставится в **LCA** с правильным путём → откат структурно корректен.
- Ранние «синтетические» коррекции (ровно 1 откат → ответ) **роняли** ассурасу ниже baseline. *Качество коррекций важнее их доли.*

Четыре стратегии поиска поверх эвристики

СТРАТЕГИЯ	КЛАССИЧЕСКИЙ АНАЛОГ	ОРАКУЛ	БЮДЖЕТ	P(SOLVE)
Greedy (temp=0)	жадный спуск	—	1	p
Majority Vote (N)	консенсус / ансамбль	—	N	$P(mode\ верен)$
Best-of-N + Verifier	random restarts + оракул	да	N	$1 - (1-p)^N$
Learned backtracking	поиск с возвратом <i>внутри</i> эвристики	—	1	$\uparrow p$

Внутренний поиск (backtracking) встроен в exp_f и не требует N независимых генераций.
Внешний (Best-of-N) применим к любой модели, но линейно дорог по compute.

Верификатор — точная проверка кандидата; reachability-солвер (DFS + мемоизация) используется как оракул допустимости. Аналогия: **отсечение** в **branch-and-bound**.

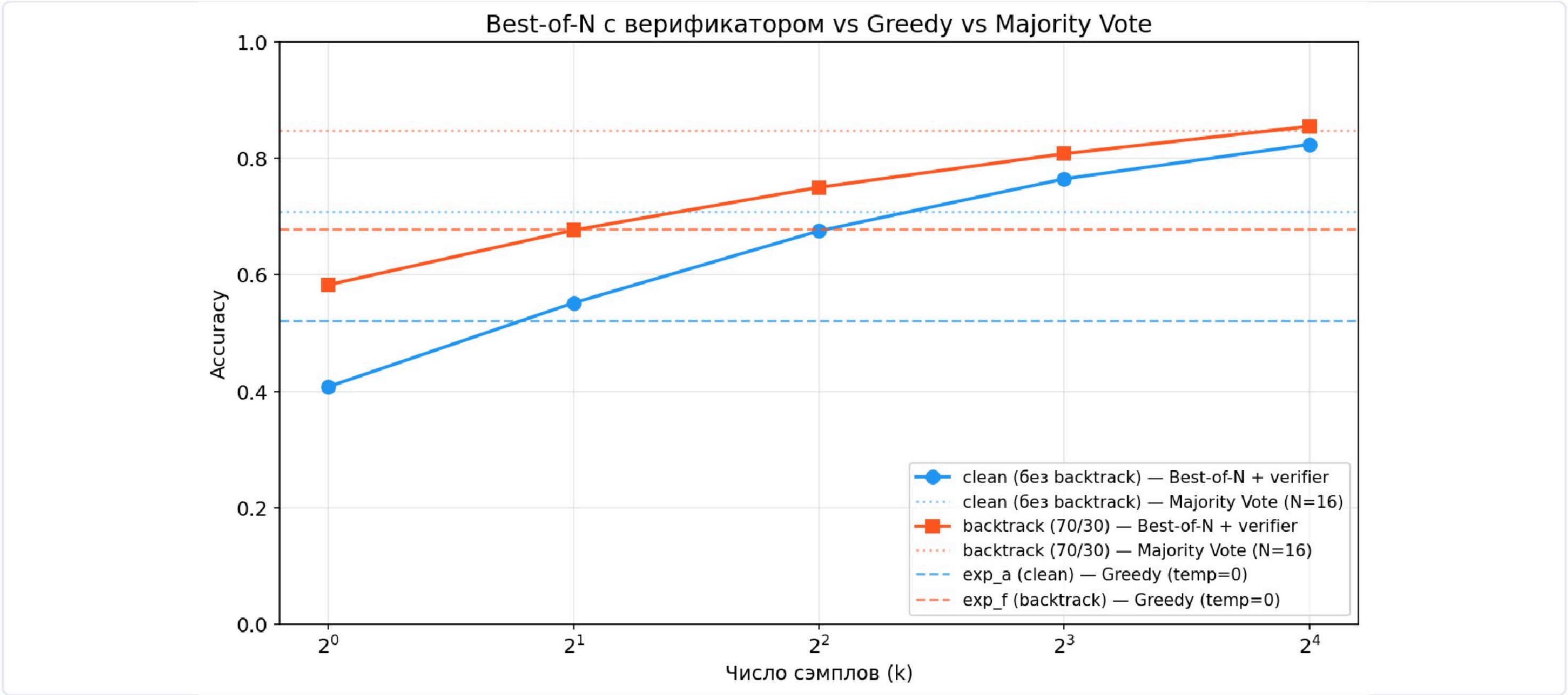
Главные числа

МОДЕЛЬ	МЕТОД	N	ACCURACY
exp_a clean	Greedy	1	52.1%
exp_f backtrack	Greedy	1	67.8% +15.7 п.п.
exp_a clean	Majority Vote	16	70.8%
exp_a clean	Best-of-16 + Verifier	16	82.4%
exp_f backtrack	Majority Vote	16	84.8%
exp_f backtrack	Best-of-16 + Verifier	16	85.5%

Один проход **exp_f (67.8%)** > exp_a (52.1%) — backtracking интернализован

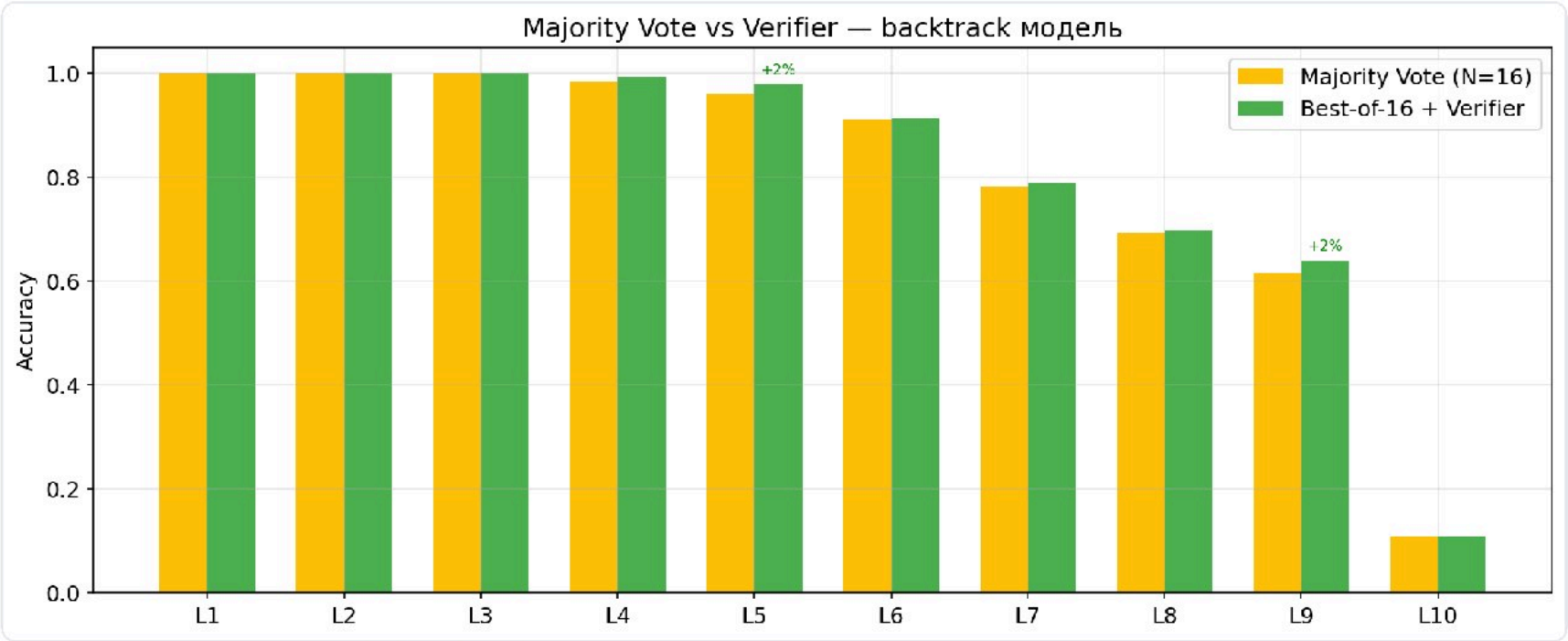
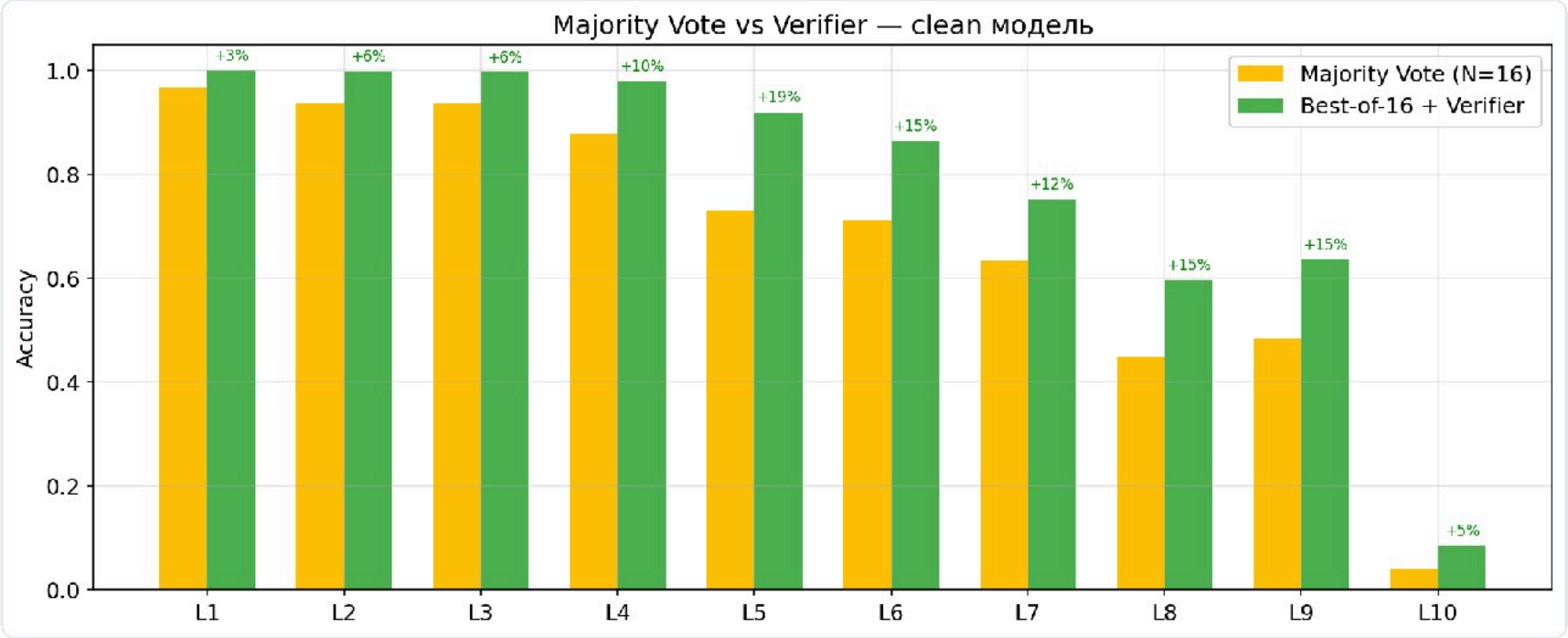
Оракул: слабой модели **+11.6 п.п.**, сильной — **+0.7 п.п.**

pass@k: внутренний поиск vs наращивание сэмплов



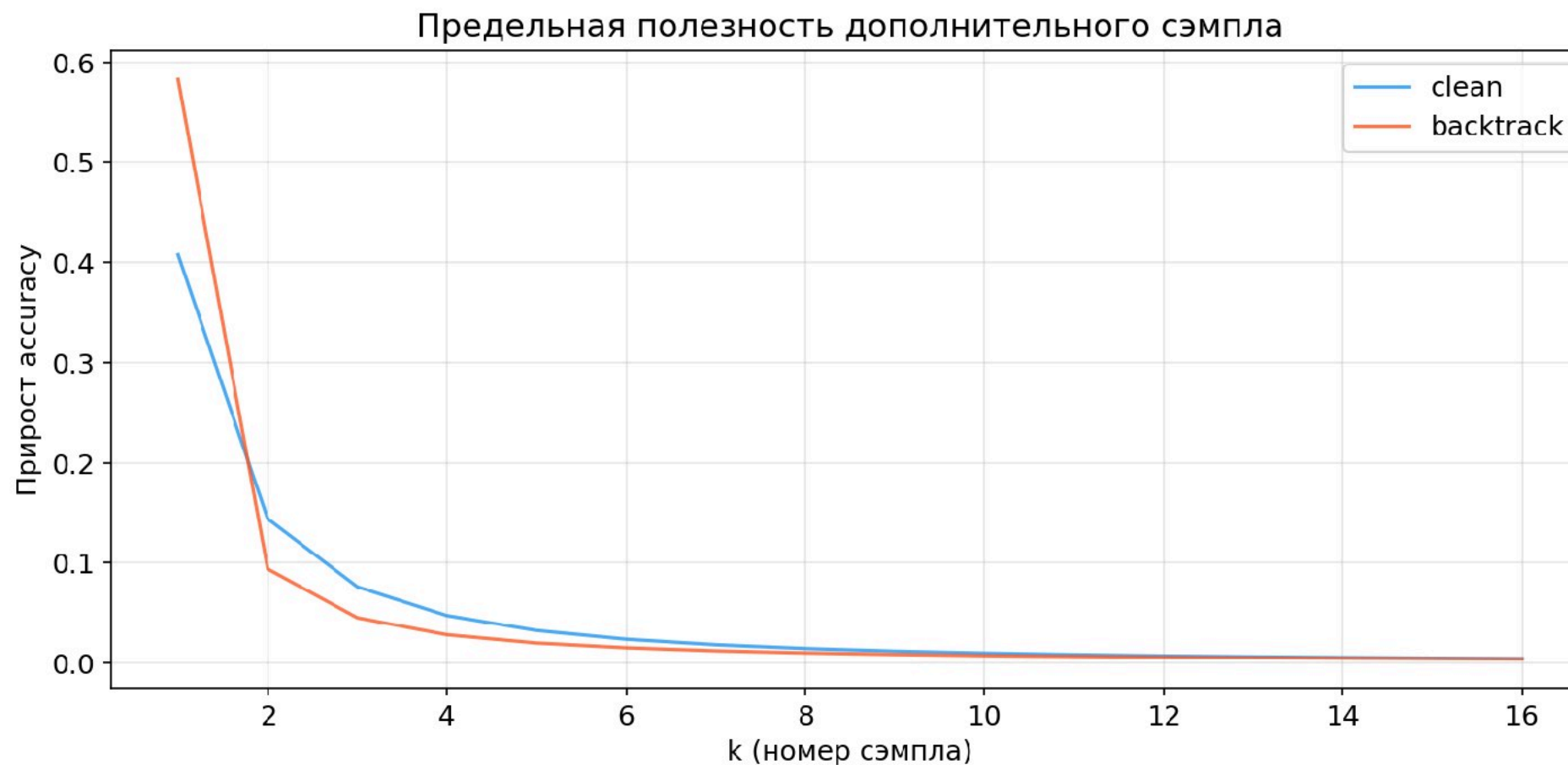
Пунктир — greedy при temp=0. Сплошные — Best-of-N + верификатор при T=0.7. Оранжевая (backtrack) кривая выше синей (clean) на всём бюджете: при одинаковом k модель с коррекциями точнее, а её greedy-уровень близок к clean + нескольким перезапускам.

Хорошая эвристика делает оракул почти избыточным



clean: +11.6 п.п. → backtrack: +0.7 п.п. — чем лучше эвристика, тем меньше пользы от внешней проверки.

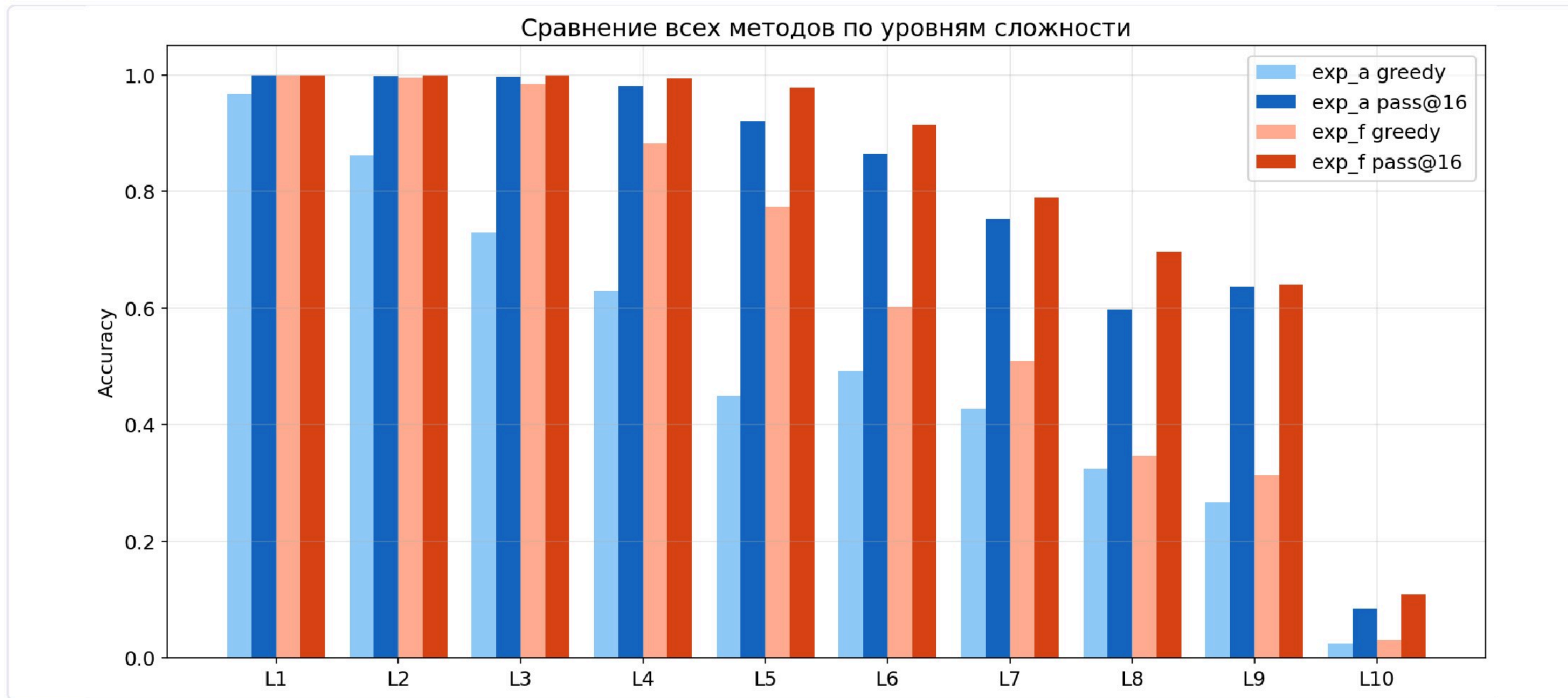
Убывающая отдача внешнего поиска



$$\partial/\partial k [1 - (1-p)^k] = -(1-p)^k \cdot \ln(1-p) \rightarrow 0$$

Каждый следующий сэмпл всё менее ценен; чем больше p , тем быстрее отдача стремится к нулю. Классическое свойство random restarts — подтверждено экспериментально.

Где это работает — по уровням сложности



L1–L3 — все методы $\approx 100\%$. **L4–L7** — максимальный выигрыш от backtracking и Best-of-N. **L8–L9** — преимущество сохраняется. **L10** — не справляется никто ($\approx 10\%$): пространство слишком велико для 16 сэмплов.

Выводы

1. Обучение на ошибках+коррекциях улучшает саму эвристику: +15.7 п.п. за один проход, без внешнего поиска.
2. Хорошая эвристика делает внешний верификатор почти избыточным: +0.7 п.п. против +11.6 п.п. у слабой модели.
3. Внешний поиск может компенсировать отсутствие **backtracking**, но дорого: 8 независимых генераций против одной траектории с возвратами.
4. Методы комплементарны: лучший результат — `exp_f` + Best-of-16 = 85.5%.

ГЛАВНЫЙ ТЕЗИС

Обучение эвристики на примерах
наращивать внешние перезапуски: оно

— более эффективный способ потратить `compute` в ветвящемся пространстве, чем просто
повышает точность каждого сэмпла и калибрует модель, снижая ценность внешнего оракула.

Спасибо! Вопросы?